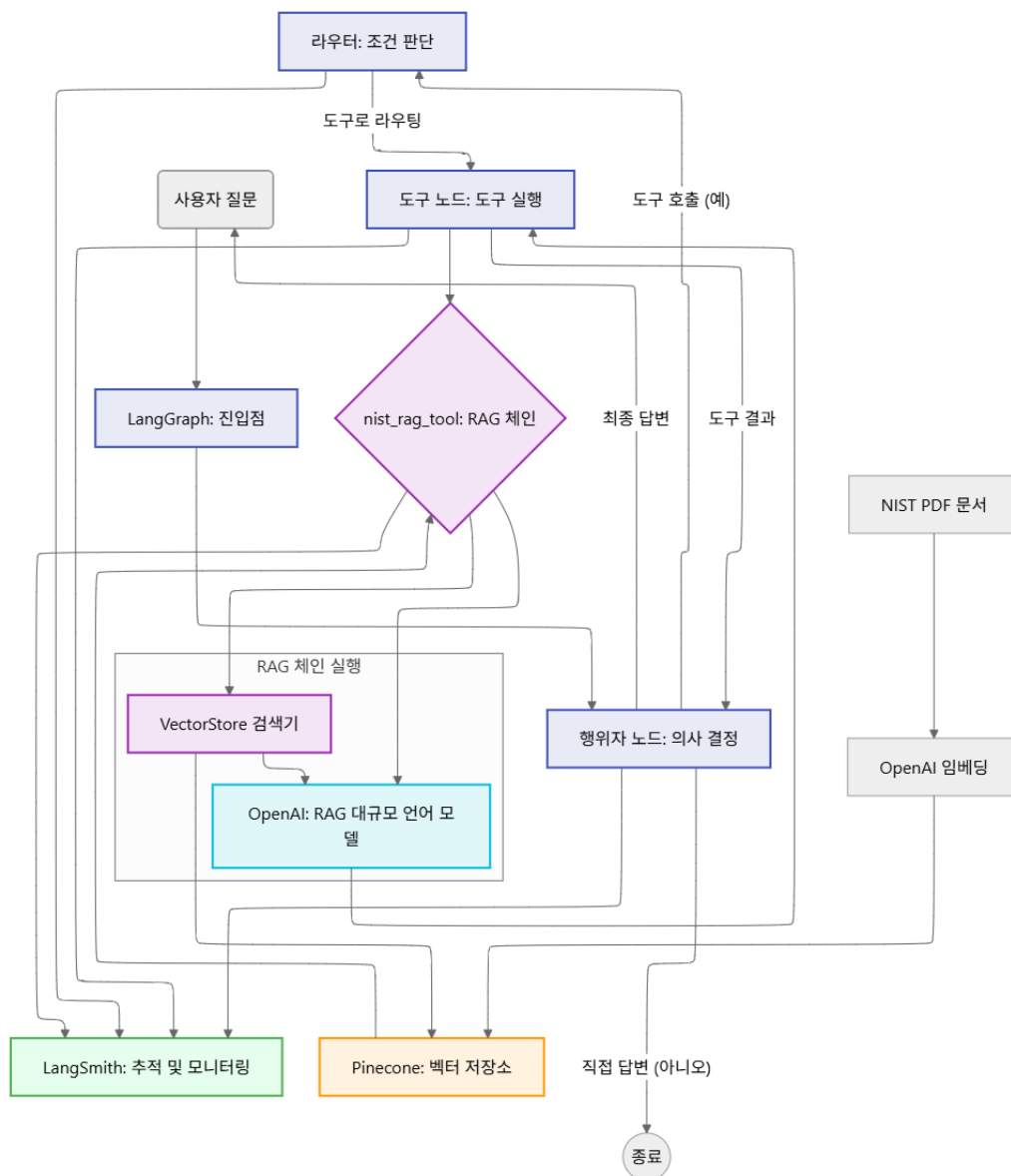


1. 서론 및 시스템 개요

1.1 서론

본 프로젝트는 NIST에서 발행한 AI 위험 관리 프레임워크(AI RMF), 사이버 보안 프레임워크(CSF 2.0), 제로 트러스트(Zero Trust) 문서들을 기반으로 지식 검색 및 컨설팅 시스템을 구축하는 것을 목표로 한다. 주요 목표는 두 가지이다. 첫째, LangChain RAG(검색 증강 생성) 체인을 구현하여 문서 근거 기반의 정확한 질의응답 시스템을 구축하였다. 둘째, LangGraph를 활용하여 이 RAG 체인을 Tool로 사용하는 대화형 Agent(컨설턴트) 시스템을 확장하고, 그 실행 과정을 LangSmith를 통해 추적 및 분석하였다.

1.2 시스템 구조 다이어그램



2. 시스템 구현 및 LangSmith 분석

2.1 RAG 체인 구현 요약

2.1.1 데이터 준비 및 청크 분할

NIST PDF 3종(ai_rmf, csf_2_0, zero_trust)을 LangChain의 PyPDFLoader를 이용해 로드하였다. 텍스트 분할 시에는 RecursiveCharacterTextSplitter를 사용하여 900자 내외로 청크를 분할하였다. 각 청크에는 향후 필터링 검색을 위해 source, page, doc_title 메타데이터를 필수로 포함하였다.

2.1.2 Pinecone 인덱스 구축 및 업서트

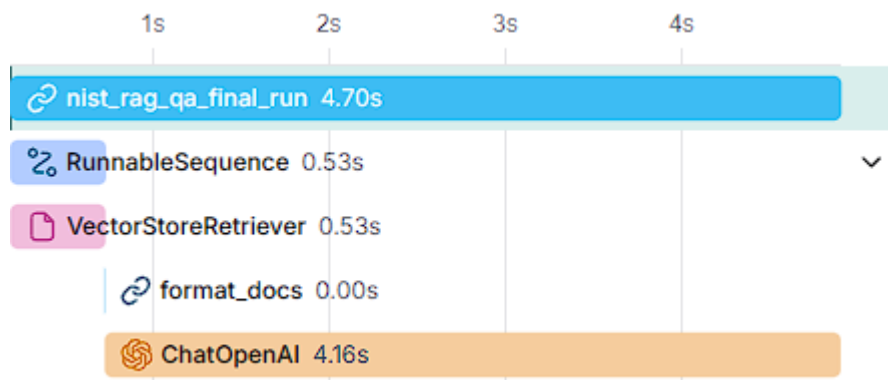
- 임베딩 모델: OpenAI의 text-embedding-3-small 모델을 사용하여 텍스트 청크를 고차원 벡터로 변환하였다.
- 인덱스 생성: Pinecone에 nist-rag-index를 Serverless 스펙으로 생성하고, 생성된 벡터들을 해당 인덱스에 업서트(UPSERT) 하였다.
- 검색기(Retriever): 업서트된 벡터스토어를 기반으로 k=4로 설정된 retriever를 구성하여, 질문과 가장 연관성 높은 4개의 문맥 청크를 검색할 수 있도록 준비하였다.

2.1.3 RAG 체인 구성

최종 RAG 체인은 LCEL(LangChain Expression Language)을 사용하여 (Retriever | Prompt | LLM) 구조로 구성하였다. Prompt에는 시스템 메시지를 통해 반드시 문서 근거를 답변에 포함하도록 명시하여, 답변의 신뢰도를 높였다. 이 완성된 RAG 체인은 2단계에서 nist_rag_tool이라는 단일 도구로 래핑되어 Agent 시스템에 통합되었다.

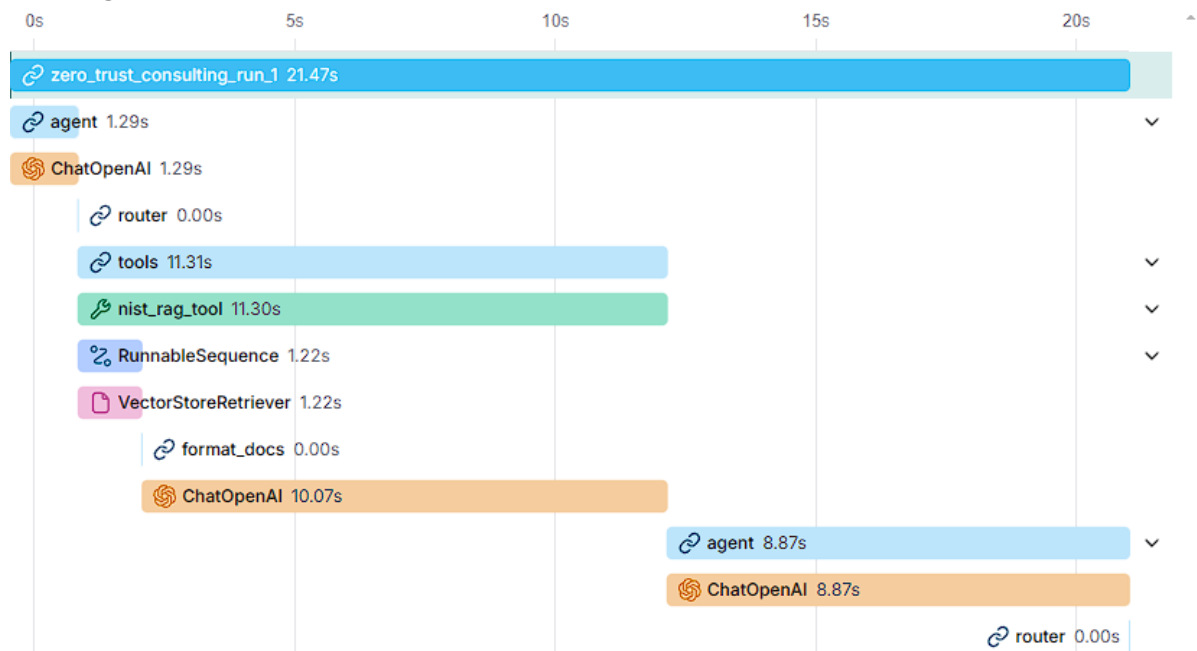
2.2 LangSmith Trace 분석

2.2.1 RAG Chain Trace



이 Trace는 검색기(Retriever)와 생성기(LLM)가 순차적으로 실행되며 RAG 체인이 오류 없이 작동했음을 증명한다. 전체 실행 시간 중 대부분(약 88%)은 LLM의 최종 답변 생성에 소요되었으며, 문서 검색 및 로딩은 매우 빠름(0.53초)을 확인하였다. 이는 일반적인 RAG 체인의 성능 특성을 보여준다

2.3.2 Agent + Graph Trace



실행 시간의 대부분(약 10초 + 8.8초)은 두 번의 LLM 호출(Tool 사용 결정 후 RAG 생성, Tool 결과 수령 후 최종 응답)에 집중되었다. LangGraph는 Agent의 결정에 따라 Tool을 정확히 호출하고 결과를 받아 처리하는 복잡한 의사 결정 플로우를 오류 없이 제어했으

며, 이는 과제의 핵심 목표를 달성했음을 의미한다.

3. 결과 평가 및 향후 계획

3.1 Agent의 Tool 사용 분석

구축된 Agent는 LangGraph의 조건부 라우팅(Conditional Routing) 덕분에 Tool 사용 여부를 성공적으로 판단하였다.

- 성공적인 Tool 호출 (NIST 관련 질문): "Zero Trust의 핵심 목표는 무엇인가요?"와 같은 NIST 문서 내의 전문 용어를 포함하는 질문에 대해서는 **Prompt 규칙(Tool 사용 강제)**에 따라 agent 노드에서 tool_calls를 생성하고, router를 거쳐 tools 노드로 진입하였다. 이는 Agent가 외부 지식이나 환각 없이 문서 근거에 기반한 답변을 제공하도록 설계된 목표를 달성했음을 의미한다.
- 성공적인 Tool 회피 (일반 질문): "오늘 점심 메뉴 추천"과 같은 NIST와 무관한 질문에 대해서는 agent 노드에서 tool_calls를 생성하지 않아, router가 흐름을 즉시 END로 종료시켰다. 이를 통해 불필요하게 RAG 체인을 가동하는 API 호출 비용과 지연 시간을 효과적으로 절감하였다.

3.2 자기 평가

3.2.1 잘 된 점

- LangGraph를 통한 제어 흐름 완성: 복잡한 RAG 체인을 nist_rag_tool이라는 하나의 도구로 추상화하고, LangGraph를 통해 Agent의 의사 결정(Tool 호출)과 실행을 명확하게 분리할 수 있었다. AgentState에 add_messages 리듀서를 적용하여 상태 관리 및 대화 연속성을 성공적으로 확보하였다.
- 투명한 디버깅 및 분석: LangSmith를 통합하여 RAG 체인의 검색 시간부터 Agent의 노드 이동까지 모든 과정을 시각적으로 추적할 수 있었으며, 이는 시스템 로직 검증과 디버깅에 결정적인 역할을 하였다.
- 근거 기반 답변: Tool의 출력 포맷을 근거 포함 텍스트로 강제하고 Agent에게 이를 편집 없이 전달하도록 지시하여, 답변의 신뢰도를 높였다.

3.2.2 아쉬운 점

- 초기 디버깅 난이도: LangGraph의 상태 관리(TypedDict와 add_messages) 및 메시지 구조(ToolMessage 역할)에 대한 이해 부족으로 인해 초기 BadRequestError와 같은 디버깅에 많은 시간이 소요되었다.
- Tool 호출 지연: Agent가 Tool을 호출하는 과정(Agent LLM 호출 + RAG Chain 실행 + Agent LLM 재호출)으로 인해 단순 RAG Chain보다 응답 시간이 현저히 길어졌다 (약 20 초 이상 소요).
- Tool 사용 효율성 한계: Agent의 Tool 사용 결정이 LLM의 Prompt 해석에만 의존하기 때문에, 질문의 키워드가 모호할 경우 잘못된 판단을 내릴 가능성이 존재한다.

3.3 향후 개선 계획

- Prompt 보완: "단순 정의 질문(What is X?)"에 대해서는 RAG Tool을 사용하도록 강제하되, "사소한 질문"에 대해서는 내부 지식으로 답변하도록 규칙의 경계를 더욱 명확하게 설정할 계획이다.
- Tool 출력 최적화: nist_rag_tool의 출력이 너무 길 경우, Agent가 이를 사용자 친화적으로 요약하도록 Prompt에 지시를 추가하여 사용자 경험을 개선할 예정이다. (단, 근거 문장은 반드시 유지)
- 성능 최적화: Agent가 Tool을 호출한 후 최종 답변 생성 단계(2차 LLM 호출)에서 지연 시간이 발생하므로, 해당 단계의 Prompt를 간소화하거나, LLM의 스트리밍 응답을 활용하여 체감 응답 속도를 개선할 계획이다.
- 필터링 기능 확장: LangGraph router 단계에서 메타데이터 필터링 기능을 추가하여, 질문이 "Zero Trust" 관련일 경우 Pinecone 검색을 {"source": "zero_trust"}로 제한하여 RAG의 정확도와 검색 속도를 향상시킬 계획이다.